`intentlock-sdk` (v0.1.0) — PyPI Installation & End-to-End Verification Report

> **Date:** November 12, 2026
> **Package Name:** `intentlock-sdk`
> **Import Module:** `intentlock`
> **Environment:** Windows, Python 3.14, isolated virtual environment (`venv`)

Executive Summary

To verify the public distribution of **`intentlock-sdk` (v0.1.0)** on PyPI, an end-to-end installation and functional test was conducted in an isolated Python 3.14 virtual environment outside the main source directory. Running `pip install intentlock-sdk` cleanly downloaded the published wheel and automatically resolved all required `pydantic` dependencies into the environment's `site-packages` . A functional smoke test script (`test_run.py`) was then executed to validate core runtime behavior by configuring a fail-closed `Policy` and `SecureAgent` , registering two tools with distinct permission and sensitivity levels (`read_report` and `drop_database`), and triggering live tool executions. The SDK successfully permitted and executed the authorized `read_report` action (`ALLOW` , `POLICY_MATCH` , risk score `0.05`), blocked the unauthorized `drop_database` action by raising a `PermissionDeniedError` (`DENY` , `PERMISSION_DENIED` , risk score `0.45`), and recorded both decisions in a structured JSON audit trail complete with UTC timestamps, policy identifiers, and SHA-256 action hashes.

1. Objective

Verify that `intentlock-sdk` (`0.1.0`) published to PyPI installs cleanly in a fresh, isolated virtual environment outside the local source directory, resolves all dependencies (`pydantic>=2.0`), and enforces security policies, tool execution, and audit logging as expected.

2. Environment Setup & Installation Steps

1. **Created an isolated test workspace (`tata`)** outside the source repository to prevent Python from importing local source files instead of the published package:

   ```
   cd ~\OneDrive\Desktop   ata
   ```

2. **Configured PowerShell Execution Policy & Virtual Environment:**

   ```
   Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
   python -m venv venv
   . env\Scripts\Activate.ps1
   ```

3. **Installed `intentlock-sdk` directly from PyPI:**

   ```
   pip install intentlock-sdk
   ```

   Result: Downloaded `intentlock_sdk-0.1.0-py3-none-any.whl` and automatically resolved `pydantic` (`2.14.3`), `pydantic-core` (`2.50.4`), `annotated-types` (`0.8.0`), `typing-extensions` (`4.16.0`), and `typing-inspection` (`0.4.5`).

4. **Verified Package Load Path:**

   ```
   python -c "import intentlock; print(intentlock.__file__)"
   ```

   Result: Confirmed module loaded directly from `... ata env\Lib\site-packages\intentlock\__init__.py`.

3. End-to-End Smoke Test Script (`test_run.py`)

```
from intentlock import (
    Config,
    Policy,
    RiskLevel,
    SecureAgent,
)


def read_report(filename: str) -> str:
    return f"Contents of {filename}: [OK]"


def drop_database(db_name: str) -> str:
    return f"Dropped {db_name}"


def run_end_to_end_test():
    # 1. Define security policy rules
    policy = Policy(
        allowed_permissions=["read_files"],
        denied_permissions=["delete_db"],
        max_risk_level=RiskLevel.MEDIUM,
        max_sensitivity=5,
        description="End-to-end verification policy",
    )

    # 2. Initialize Config (fail-closed + audit enabled) and SecureAgent
    config = Config(default_policy=policy, fail_closed=True, audit_enabled=True)
    agent = SecureAgent(
        agent_id="e2e-test-agent",
        permissions=["read_files", "delete_db"],
        config=config,
    )

    # 3. Register tools with handlers and sensitivity ratings
    agent.register_tool(
        name="read_report",
        handler=read_report,
        permission="read_files",
        description="Safely read a report file",
        sensitivity=1,
    )
    agent.register_tool(
        name="drop_database",
        handler=drop_database,
        permission="delete_db",
        description="Delete a database",
        sensitivity=9,
    )

    # 4. Test 1: Execute allowed tool
    print("--- Test 1: Executing allowed tool (read_report) ---")
    try:
        result = agent.execute(
            tool_name="read_report",
            arguments={"filename": "summary.txt"},
```

```
            user_request="Read the summary report",
        )
        print(f"SUCCESS -> Result: {result}
")
    except Exception as exc:
        print(f"UNEXPECTED ERROR -> {type(exc).__name__}: {exc}
")

    # 5. Test 2: Execute denied tool
    print("--- Test 2: Executing denied tool (drop_database) ---")
    try:
        result = agent.execute(
            tool_name="drop_database",
            arguments={"db_name": "prod_db"},
            user_request="Drop the production database",
        )
        print(f"RESULT -> {result}
")
    except Exception as exc:
        print(f"BLOCKED AS EXPECTED -> {type(exc).__name__}: {exc}
")

    # 6. Verify audit log entries
    print("--- Audit Log ---")
    for entry in agent.get_audit_log():
        print(entry)


if __name__ == "__main__":
    run_end_to_end_test()
```

4. Verification Results

Test Case	Component Tested	Expected Outcome	Actual Outcome
Package Install	`pip install intentlock-sdk`	Downloads wheel and `pydantic` dependencies cleanly	Installed `intentlock-sdk-0.1.0` + dependencies in `venv`
Tool Registration	`SecureAgent.register_tool()`	Registers handlers with permission & sensitivity metadata	Logged structured JSON event `intentlock.registry`
Allowed Execution	`SecureAgent.execute("read_report")`	Evaluates `ALLOW` (`POLICY_MATCH`) and executes handler	Returned `Contents of summary.txt: [OK]` (`risk_score=0.05`)
Denied Execution	`SecureAgent.execute("drop_database")`	Evaluates `DENY` (`PERMISSION_DENIED`) and blocks handler	Raised `PermissionDeniedError` (`risk_score=0.45`)
Audit Trail	`SecureAgent.get_audit_log()`	Records tamper-evident `AuditEvent` objects for both calls	Recorded 2 events with UTC timestamps, verdicts, and SHA `action_hash`